

IncidentFlow

Spécification Détaillée de l'Architecture & du Flux de Données

Des Entités Base de Données à l'Interface Utilisateur Real-Time

Synthèse du Document

Ce document présente le fonctionnement interne du système **IncidentFlow**. Il détaille le parcours complet des données relatif aux **Utilisateurs**, aux **Rôles**, et au **Workflow Dynamique** (États et Transitions), depuis la persistance en base de données PostgreSQL jusqu'à l'affichage et la synchronisation en temps réel sur le frontend React.

Application :	IncidentFlow (Gestion d'Incidents Interne)
Stack Backend :	Java 17, Spring Boot 3, JPA / Hibernate, PostgreSQL
Stack Frontend :	React 19, Vite, TailwindCSS / Vanilla CSS
Auteur :	Équipe de Développement NetMar
Date :	23 juillet 2026

Table des matières

1	Vue d'Ensemble de l'Architecture 3-Tiers	2
2	Couche 1 : Base de Données & Modèle Objet JPA	2
2.1	Gestion des Utilisateurs et des Rôles	2
2.2	Modélisation du Workflow Dynamique (États et Transitions)	3
2.3	Initialisation des Données en Base (DataInitializer.java)	3
3	Couche 2 : Couche Métier & REST Backend (Spring Boot 3)	3
3.1	Accès aux Données (Repositories)	3
3.2	Couche Services & Gestion des Entités Détachées JPA	3
3.2.1	Résolution du Bug Détaché JPA dans WorkflowService.java	4
3.3	Contrôleurs REST (API Endpoints)	4
4	Couche 3 : Interface Utilisateur Frontend (React 19 / Vite)	4
4.1	Chargement Initial et Reconstitution des États (App.jsx)	4
4.2	Propagation en Temps Réel dans l'Interface (Real-Time UI Sync)	5
5	Scénario Complet : De la Modification en Base au Rendu Temps Réel	5
6	Tableau Récapitulatif des Fichiers du Projet	6

1 Vue d'Ensemble de l'Architecture 3-Tiers

L'application **IncidentFlow** repose sur une architecture robuste à trois tiers. Les composants interagissent de manière découplée via des API REST normées et du JSON.

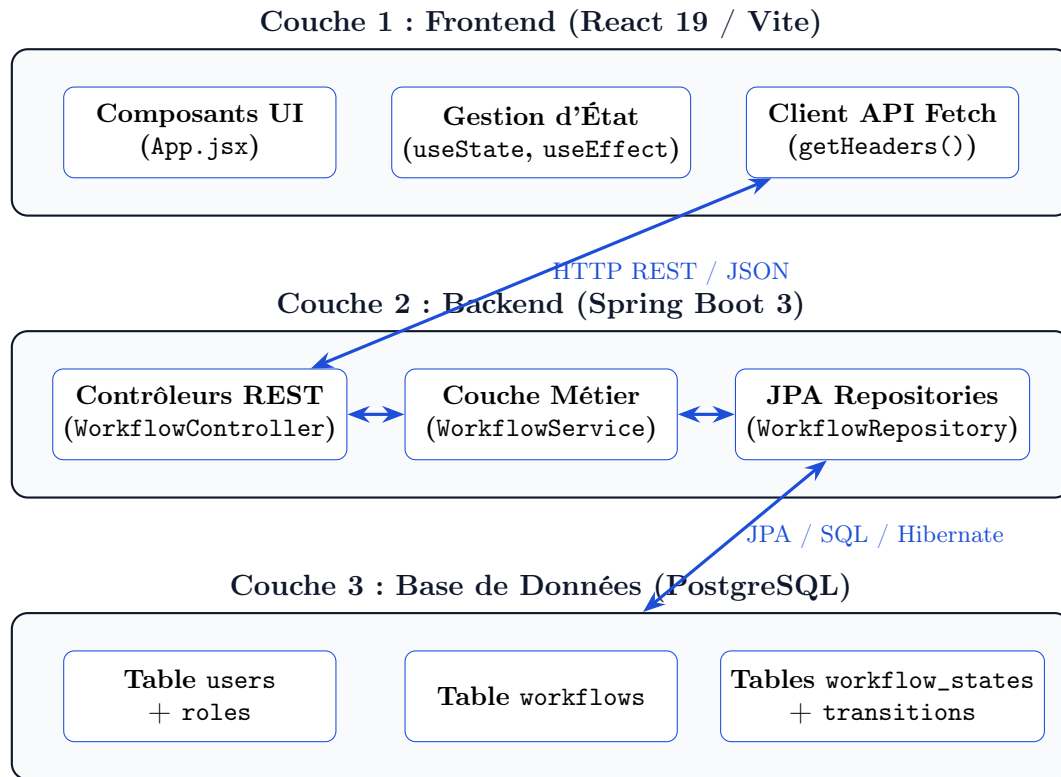


FIGURE 1 – Diagramme de flux de données de bout en bout de l'application IncidentFlow

2 Couche 1 : Base de Données & Modèle Objet JPA

La persistance des données s'appuie sur le SGBD **PostgreSQL**. La modélisation est représentée côté Java par des entités JPA (Java Persistence API) annotées avec Hibernate.

2.1 Gestion des Utilisateurs et des Rôles

Les utilisateurs et les rôles sont modélisés par deux entités liées par une relation de type `@ManyToOne`.

- **Fichier Rôle** : `backend/src/main/java/com/netmar/incidentflow/model/Role.java`
- **Fichier Utilisateur** : `backend/src/main/java/com/netmar/incidentflow/model/User.java`

```
@Entity
@Table(name = "users")
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String email;

    private String name;

    @ManyToOne
    @JoinColumn(name = "role_id")
```

```
private Role role; // Contient "Administrateur", "Responsable", "Opérateur",
    etc.
}
```

Listing 1 – Structure JPA de l'entité User et sa relation avec Role

2.2 Modélisation du Workflow Dynamique (États et Transitions)

Le moteur de workflow est composé de trois entités en cascade : Workflow, WorkflowState et WorkflowTransition.

- **Fichier d'Entité** : backend/src/main/java/com/netmar/incidentflow/model/Workflow.java
- **Fichier États** : backend/src/main/java/com/netmar/incidentflow/model/WorkflowState.java
- **Fichier Transitions** : backend/src/main/java/com/netmar/incidentflow/model/WorkflowTransition.java

```
@Entity
@Table(name = "workflows")
public class Workflow {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToMany(mappedBy = "workflow", cascade = CascadeType.ALL, orphanRemoval =
        true, fetch = FetchType.EAGER)
    private List<WorkflowState> states = new ArrayList<>();

    @OneToMany(mappedBy = "workflow", cascade = CascadeType.ALL, orphanRemoval =
        true, fetch = FetchType.EAGER)
    private List<WorkflowTransition> transitions = new ArrayList<>();
}
```

Listing 2 – Association en Cascade et Orphelins du Workflow

2.3 Initialisation des Données en Base (DataInitializer.java)

Au démarrage du serveur backend, la classe d'initialisation vérifie l'existence des rôles et du workflow standard.

- **Fichier Source** : backend/src/main/java/com/netmar/incidentflow/config/DataInitializer.java

Si aucun workflow n'est présent (`workflowRepository.count() == 0`), un workflow par défaut nommé **Workflow Standard** est créé avec 5 états principaux (*Nouveau*, *Assigné*, *En cours*, *Résolu*, *Clôturé*) et leurs transitions respectives.

3 Couche 2 : Couche Métier & REST Backend (Spring Boot 3)

La couche backend assure la sécurité, la validation métier, le traitement des entités détachées JPA et l'exposition des API REST.

3.1 Accès aux Données (Repositories)

Les interfaces Spring Data JPA permettent d'exécuter les requêtes SQL nécessaires :

- UserRepository.java : `findByEmail(String email)`
- RoleRepository.java : `findByName(String name)`
- WorkflowRepository.java : `findByCategory(String category)`

3.2 Couche Services & Gestion des Entités Détachées JPA

La logique métier est centralisée dans `UserService.java` et `WorkflowService.java`.

3.2.1 Résolution du Bug Détaché JPA dans WorkflowService.java

Lorsqu'un utilisateur modifie un workflow depuis le frontend, les objets JSON reçus contiennent des identifiants existants d'états mais ne sont pas rattachés à la session Hibernate courante. Un `repository.save()` direct provoquait l'erreur `detached entity passed to persist`.

— **Fichier Modifié :** `backend/src/main/java/com/netmar/incidentflow/service/WorkflowService.java`

```
@Transactional
@CacheEvict(value = {"workflows", "workflows-category"}, allEntries = true)
public Workflow saveWorkflow(Workflow workflow) {
    if (workflow.getId() != null) {
        // 1. Charger l'entite geree par JPA
        Workflow existing = workflowRepository.findById(workflow.getId()).
            orElseThrow();
        existing.setName(workflow.getName());

        // 2. Vider les collections (declenche orphanRemoval = true)
        existing.getStates().clear();
        existing.getTransitions().clear();

        // 3. Re-instancier et rattacher les nouveaux enfants a l'entite geree
        if (workflow.getStates() != null) {
            for (WorkflowState s : workflow.getStates()) {
                existing.getStates().add(WorkflowState.builder()
                    .name(s.getName()).label(s.getLabel())
                    .colorClass(s.getColorClass()).workflow(existing).build());
            }
        }
        return workflowRepository.save(existing);
    }
    return workflowRepository.save(workflow);
}
```

Listing 3 – Résolution de la persistance des entités détachées

3.3 Contrôleurs REST (API Endpoints)

Les données sont exposées au frontend via les contrôleurs REST :

Méthode	Endpoint API	Contrôleur Backend	Description
GET	<code>/api/users</code>	<code>UserController.java</code>	Récupère tous les utilisateurs
GET	<code>/api/roles</code>	<code>UserController.java</code>	Récupère la liste des rôles
GET	<code>/api/workflows</code>	<code>WorkflowController.java</code>	Récupère le workflow unique
PUT	<code>/api/workflows/{id}</code>	<code>WorkflowController.java</code>	Enregistre les modifications du workflow
DELETE	<code>/api/incidents/{code}</code>	<code>IncidentController.java</code>	Supprime un incident (Admin requis)

TABLE 1 – Principaux points de terminaison REST de l'application

4 Couche 3 : Interface Utilisateur Frontend (React 19 / Vite)

Le frontend React consomme l'API REST via la fonction système `fetch` et met à jour dynamiquement l'interface grâce au système d'états réactifs de React.

4.1 Chargement Initial et Reconstitution des États (App.jsx)

Au chargement du composant principal `App.jsx`, le hook `useEffect` déclenche les requêtes asynchrones pour récupérer les workflows, les utilisateurs et les incidents.

— Fichier Frontend Principal : frontend/src/App.jsx

```
const [workflows, setWorkflows] = useState([]);
const [activeWorkflow, setActiveWorkflow] = useState(null);

const fetchWorkflows = async () => {
  try {
    const res = await fetch(`${API_BASE}/workflows`, { headers: getHeaders() });
    if (res.ok) {
      const data = await res.json();
      setWorkflows(data);
      if (data.length > 0) setActiveWorkflow(data[0]);
    }
  } catch (err) { console.error(err); }
};
```

Listing 4 – Appels asynchrones et synchronisation d'état React

4.2 Propagation en Temps Réel dans l'Interface (Real-Time UI Sync)

Chaque modification du workflow (ajout/suppression d'un état ou d'une transition) est immédiatement propagée sur trois composants clés de l'interface :

1. **Éditeur Graphique de Workflow** : L'administrateur modifie les nœuds et les connexions dans l'éditeur interactif.
2. **Filtre de Statut Dynamique dans la Liste des Incidents** : Le menu déroulant de sélection du statut s'alimente directement depuis `activeWorkflow.states` :

```
<select value={statusFilter} onChange={(e) => setStatusFilter(e.target.value)}>
  <option value="Tous">Tous</option>
  {activeWorkflow?.states.map(s => (
    <option key={s.id || s.name} value={s.name}>{s.label || s.name}</option>
  ))}
</select>
```

3. **Stepper Horizontal & Boutons d'Action (Fiche de Détail)** : Le stepper visuel d'avancement ainsi que les boutons d'action autorisés (*Passer à : En cours*, etc.) s'évaluent dynamiquement en fonction des états du workflow actif :

```
const currentWf = activeWorkflow || workflows[0];
// Rendu dynamique du Stepper Horizontal et des Actions Disponibles
```

5 Scénario Complet : De la Modification en Base au Rendu Temps Réel

Pour illustrer le passage complet des données à votre encadrant, voici les 6 étapes de l'exécution complète lorsqu'un Administrateur ajoute un état au workflow :

Parcours d'une Modification en 6 Étapes

1. **Action Utilisateur (Frontend) :** L'administrateur ajoute un nouvel état (ex : *En Validation*) dans la page de gestion du Workflow et clique sur "*Enregistrer les modifications*".
2. **Requête HTTP PUT (Client REST) :** React envoie une requête PUT /api/workflows/1 avec le payload JSON structuré contenant les nouveaux états et transitions.
3. **Réception & Sécurité (Backend) :** WorkflowController.java reçoit la requête, transmet l'objet à WorkflowService.java et vérifie le rôle de l'utilisateur.
4. **Traitement JPA & Persistance SQL (Service & Database) :**
 - WorkflowService charge l'entité gérée existing.
 - Effacement des anciennes collections via .clear() (déclenchant les DELETE SQL).
 - Insertion des nouveaux états via save() (déclenchant les INSERT SQL dans workflow_states).
5. **Réponse HTTP 200 & Mise à jour du State React :** Le backend renvoie l'objet Workflow mis à jour. App.jsx met à jour son état React local via setActiveWorkflow(updated) et déclenche un fetchIncidents().
6. **Mise à Jour Visuelle Temps Réel (DOM React) :** React ré-évalue le composant. Le nouvel état "*En Validation*" apparaît instantanément dans :
 - Le menu déroulant du filtre de statut de la page d'incidents.
 - Le stepper horizontal de progression de la fiche détaillée d'incident.
 - Le panneau des actions de transition disponibles.

6 Tableau Récapitulatif des Fichiers du Projet

Composant / Fichier	Emplacement dans le Projet	Rôle Principal dans le Flux
User.java	backend/.../model/User.java	Entité JPA décrivant l'utilisateur
Role.java	backend/.../model/Role.java	Entité JPA décrivant les rôles/permissions
Workflow.java	backend/.../model/Workflow.java	Entité racine du moteur de workflow
WorkflowState.java	backend/.../model/WorkflowState.java	Entité enfant décrivant chaque état
WorkflowTransition.java	backend/.../model/WorkflowTransition.java	Entité enfant décrivant les transitions
DataInitializer.java	backend/.../config/DataInitializer.java	Script de seeding initial au démarrage
UserService.java	backend/.../service/UserService.java	Gestion métier utilisateurs/rôles
WorkflowService.java	backend/.../service/WorkflowService.java	Logique de persistance et re-attachement
WorkflowController.java	backend/.../controller/WorkflowController.java	Endpoints REST pour le workflow
IncidentController.java	backend/.../controller/IncidentController.java	Endpoints REST incidents + suppression
App.jsx	frontend/src/App.jsx	Interface React, état global et routing

TABLE 2 – Matrice des fichiers clés du projet IncidentFlow